

GramText: Devanagari Sign & Prescription Reader for Digital Literacy in Rural Communities

(SDG 3, 4 and 10)

*A project report submitted in partial fulfilment of the requirements
for the award of the degree of*

MASTER OF COMPUTER APPLICATIONS

Submitted by

[Student Name 1] ([Roll No.]

[Student Name 2] ([Roll No.]

Under the guidance of

[Guide Name]

[Designation], Department of Computer Applications



Department of Computer Applications

[College Name]

[University Name]

Academic year 2026–27

projexa.ai Team 26040 · Approved · Community SDG

CERTIFICATE

This is to certify that the project report titled "**GramText: Devanagari Sign & Prescription Reader for Digital Literacy in Rural Communities**" is a bonafide work carried out by [Student name(s)], Roll No. [], in partial fulfilment of the requirements for the degree of Master of Computer Applications at [College name] during the academic year 2026–27. The work has not been submitted elsewhere for any other degree or diploma.

Project Guide

Head of Department

External Examiner

DECLARATION

I/We declare that this project report is my/our own original work. It was carried out under the supervision of [Guide name]. Where material from other sources has been used, it has been acknowledged in the references. This report has not been submitted to any other university or institution for any degree.

Date: _____

Place: _____

Signature of student(s)

ACKNOWLEDGEMENT

We thank our project guide, [Guide name], for steady guidance throughout this work. We thank [Head of Department] and the faculty of the Department of Computer Applications for their support and facilities. We are grateful to [NGO name] for helping us understand the needs of rural users and for supporting field testing. We also thank the open-source communities behind PyTorch, timm, docTR, FastAPI, ONNX Runtime and gTTS, whose tools made this project possible. Finally, we thank our families and friends for their encouragement.

Student(s)

ABSTRACT

GramText is an Android app that photographs printed Hindi or English text and reads it aloud. It is built for rural users with low literacy who cannot read medicine labels, prescriptions or government scheme notices printed in Devanagari.

The system has three parts. A Kotlin app built with Jetpack Compose and CameraX captures the photo. A FastAPI server finds the words with a pretrained DBNet text detector and reads each word with our own scene text recognition (STR) model. The gTTS library then turns the text into Hindi or English speech. Our STR model pairs a MobileViTv2-0.5 backbone with a two-layer BiLSTM and a CTC output layer over 213 characters. It has about 0.85 million parameters, and its ONNX file is about 3.5 MB.

The model is trained on synthetic word images. These are rendered from 20,000 Hindi and 47,000 English words in 14 fonts, shaped with HarfBuzz so that matras and conjuncts look as they do in print. Camera-like blur, noise, perspective and compression effects are added. In a CPU overfit test on 64 samples, character accuracy reached 87% within 1,000 steps, which confirms that the pipeline learns. When the model's confidence is below 0.85, or when no trained model is installed, the server falls back to Google Gemini.

The interface is Hindi-first. It has large icon buttons, 24 sp text, automatic read-aloud, a slow-speech option and screen-reader labels. A metrics endpoint records anonymous counts (scans, success rate, helpful rate, corrections, latency and unique devices) to measure impact during a field pilot with a partner NGO. No photos or scanned text are stored.

The project supports SDG 3 (Good Health), SDG 4 (Quality Education) and SDG 10 (Reduced Inequalities). The approved proposal called for an offline-first app using TFLite. This phase delivers an online client–server version, and offline inference on the phone is planned as Phase 2.

Keywords: Devanagari OCR, scene text recognition, MobileViT, CTC, text-to-speech, digital literacy, assistive technology, Sustainable Development Goals.

TABLE OF CONTENTS

Certificate	ii
Declaration	iii
Acknowledgement	iv
Abstract	v
Chapter 1 Introduction	1
1.1 Background	1
1.2 Motivation	1
1.3 Problem statement	2
1.4 Objectives	2
1.5 Scope	2
1.6 Organisation of the report	3
Chapter 2 Literature survey	4
2.1 From classical OCR to deep learning	4
2.2 Text detection	4
2.3 Text recognition and CTC	4
2.4 Lightweight vision backbones	5
2.5 Challenges of Devanagari text	5
2.6 Text-to-speech	6
2.7 Existing applications	6
2.8 Gap analysis	6
Chapter 3 Requirements analysis	7
3.1 Feasibility study	7
3.2 Stakeholders	7
3.3 Functional requirements	8
3.4 Non-functional requirements	8
3.5 Hardware requirements	9
3.6 Software requirements	9
Chapter 4 System design	10
4.1 System architecture	10
4.2 Module design	11
4.3 Data flow	12
4.4 Use cases	12
4.5 Sequence of a scan	13
4.6 API design	13

4.7 Database design	13
4.8 User interface design	14
Chapter 5 The GramText STR model	16
5.1 Character set	16
5.2 Synthetic training data	16
5.3 Preprocessing	18
5.4 Architecture	18
5.5 CTC loss and decoding	18
5.6 Training	19
5.7 Export	19
5.8 Evaluation metrics	19
Chapter 6 Implementation	21
6.1 Backend	21
6.2 Android app	23
6.3 Web demo	24
6.4 Deployment	24
6.5 Security measures	24
Chapter 7 Testing	25
7.1 Testing strategy	25
7.2 Automated backend tests	25
7.3 Model tests	26
7.4 System and device tests	26
7.5 User acceptance testing	27
Chapter 8 Results and SDG impact	28
8.1 Delivered outputs	28
8.2 Development testing results	28
8.3 Model results	29
8.4 Screenshots	29
8.5 SDG alignment	30
8.6 NGO engagement and field pilot plan	30
Chapter 9 Limitations, future work and conclusion	31
9.1 Deviations from the approved proposal	31
9.2 Limitations	31
9.3 Future work	32
9.4 Conclusion	32
References	34
Appendix A: Installation and running	36

Appendix B: Configuration	37
Appendix C: Sample API response	38
Appendix D: Model code	39
Appendix E: Viva preparation	40

CHAPTER 1

Introduction

1.1 Background

Many everyday decisions in rural India depend on reading printed text. These include medicine dosages, warnings on a strip of tablets, a doctor's printed prescription, and the eligibility rules on a government scheme notice. Most of this text is in Hindi, written in the Devanagari script, or in English.

According to the 2011 Census of India, rural literacy was about 68%, compared with about 84% in urban areas, and it was lower still among rural women. Being counted as literate does not always mean a person can read small, dense print on a medicine label. A person who cannot read the label must rely on a relative, a shopkeeper or a health worker. When help is not available, mistakes in dosage or missed benefits can follow.

Smartphones are now common in rural households, even where literacy is low. A phone camera plus speech output can close this gap: the user points the camera at the text, and the phone reads it aloud in a familiar language.

1.2 Motivation

General-purpose OCR tools exist, but they do not serve this group well, for four reasons:

- **Script difficulty.** Devanagari uses a headline (shirorekha), vowel signs (matras) above, below and beside the consonant, and conjunct consonants. Generic OCR engines often drop or misplace these marks.
- **Interface.** Most apps assume the user can read menus and results. A low-literacy user needs icons, large text, Hindi labels and automatic speech.
- **Mixed text.** Medicine labels mix Hindi and English, for example a Hindi instruction next to "Paracetamol 500 mg". The speech output must switch voices between the two.
- **Measurable impact.** A community project must show whether it helps. That needs anonymous usage and feedback data, collected without storing private photos.

1.3 Problem statement

In rural areas, low-literacy individuals cannot read medicine labels, prescriptions or government scheme notices printed in Devanagari. In partnership with an NGO working on health and literacy, this project delivers a mobile tool that captures printed Devanagari (and English) text and reads it aloud. It supports Good Health (SDG 3), Quality Education (SDG 4) and Reduced Inequalities (SDG 10).

1.4 Objectives

The approved proposal set four objectives. The table shows how this phase addresses each one.

Objective (from the proposal)	How this phase addresses it
Build offline-first, low-resource inference for rural connectivity	Low-resource: a small STR model (about 0.85M parameters) and photos shrunk to at most 2048 px (about 400 KB) before upload. Offline-first is deferred to Phase 2
Engage with NGO stakeholders and field-test with target users	NGO collaboration letter, community feedback form and impact report templates; in-app helpful/not helpful feedback with optional corrections
Design inclusive UX for low-literacy users	Hindi-first interface, large icon buttons, 24 sp text, automatic read-aloud, slow-speech option, editable text, TalkBack labels
Align technical work with measurable SDG outcomes	A metrics endpoint for scans, success rate, helpful rate, corrections, latency and unique devices, plus model accuracy reports

1.5 Scope

In scope for this phase:

- Printed Hindi (Devanagari) and English text on labels, prescriptions and notices
- An Android app, a FastAPI backend, and an optional React web demo of the same API
- Our own STR model, trained on synthetic data and exported to ONNX
- Hindi and English speech output, including mixed-language labels
- Anonymous impact metrics and user feedback

Out of scope for this phase:

- Offline, on-device recognition (TFLite) and offline speech. These are deferred to Phase 2 by the team's decision.
- Handwritten prescriptions. The model is trained on printed text only.
- Medical advice of any kind. The app reads the text and nothing more.

1.6 Organisation of the report

Chapter 2 reviews earlier work on OCR and Devanagari text recognition. Chapter 3 states the requirements. Chapter 4 presents the system design. Chapter 5 describes the STR model and how it is trained. Chapter 6 covers the implementation of the backend, the Android app and the deployment. Chapter 7 describes testing. Chapter 8 presents results and the SDG impact plan. Chapter 9 discusses limitations, future work and conclusions.

CHAPTER 2

Literature survey

This chapter reviews the methods GramText builds on: how text recognition developed, why Devanagari is hard, and which existing tools come closest to our goal. Numbers in square brackets refer to the References section.

2.1 From classical OCR to deep learning

Early optical character recognition (OCR) systems split a page into lines, words and single characters, then classified each character with hand-made features. Tesseract [1], first developed at HP and later released as open source, is the best-known example. These methods work well on clean scanned documents. They fail on photos taken with a phone, where the text is blurred, tilted, unevenly lit or printed on curved packaging.

Deep learning changed the field. Instead of cutting out each character, modern systems read a whole word image at once. This approach is called **scene text recognition (STR)**. A typical STR system has two stages:

1. **Text detection** finds where the words are in the photo.
2. **Text recognition** reads the characters in each detected word.

2.2 Text detection

EAST [2] predicts word boxes directly from a fully convolutional network. CRAFT [3] predicts how likely each pixel is to be the centre of a character, and links characters into words. DBNet [4] predicts a probability map of text pixels and learns the threshold that separates text from background (differentiable binarisation). DBNet is fast and accurate, and the docTR library [5] provides pretrained DBNet models. GramText uses the docTR DBNet detector without retraining.

2.3 Text recognition and CTC

CRNN [6] combined a convolutional network (to extract visual features), a recurrent network (to model the order of characters) and a **Connectionist Temporal Classification (CTC)** output layer [7]. CTC solves a key problem: training images are labelled with a word, but not with the position of each character. CTC adds a special "blank" symbol and

sums over every possible alignment of the label to the image columns. At inference time, greedy decoding takes the most likely symbol at each step, merges repeats and removes blanks.

Attention-based decoders [8] and transformer recognisers such as PARSeq [9] reach higher accuracy on English benchmarks. They are larger and slower, and they decode one character at a time. For a small, fast model on a low-cost server, a CNN–RNN–CTC design remains a strong choice. GramText follows this design but replaces the plain CNN with MobileViT.

2.4 Lightweight vision backbones

MobileNet [10] made convolutional networks practical on phones by using depthwise separable convolutions. Vision Transformers (ViT) [11] model long-range context across the whole image, but they need large datasets and much computation. **MobileViT** [12] combines both: convolution layers capture local strokes, and small transformer blocks capture global context. **MobileViTv2** [13] replaces the costly self-attention with separable self-attention, which is linear in the number of tokens. The smallest version, MobileViTv2-0.5, has about 1.4M parameters for ImageNet classification. We use only its first stages, so our full recogniser stays under 1M parameters.

2.5 Challenges of Devanagari text

Devanagari is an abugida: each consonant carries an inherent vowel, and other vowels are written as signs (matras) attached to it. The features that make it hard to recognise are:

- **Shirokekha.** A headline joins the letters of a word, so characters cannot be separated by simple gaps.
- **Matras on all sides.** Vowel signs sit above (ँ, ॐ), below (ॐ, ॐ), before (ॐ) or after (ॐ, ॐ) the consonant. In Unicode the short-i sign (ॐ) is stored after the consonant but drawn before it.
- **Conjuncts.** Consonant clusters such as क्ष (ksha) and त्र (tra) are drawn as one combined shape, while Unicode stores them as several code points joined by a virama.
- **Fonts.** Printed labels use many fonts, and each draws conjuncts differently.

Earlier Devanagari OCR research segmented characters and classified them individually, with modest accuracy on real photos [14]. The IIIT-ILST dataset [15] provided real scene-text images in Hindi, Telugu and Malayalam, and showed that CRNN models trained on synthetic data can transfer to real photos. IndicSTR12 [16] extended this to 12 Indian languages. Both works found that synthetic data must be rendered with correct text shaping. Otherwise, matras and conjuncts appear in the wrong positions.

GramText addresses this by recognising text at the Unicode code-point level and rendering every training image with HarfBuzz [17], the shaping engine used by browsers and Android itself.

2.6 Text-to-speech

Android's built-in TTS supports Hindi on many phones, but voice quality and availability vary by device and manufacturer. Neural TTS models give natural speech but are heavy to host. gTTS [18] is a Python library that uses the Google Translate speech service. It supports Hindi and Indian-accented English, needs no API key, and returns MP3 audio. Its limits are that it needs a network and has no service-level agreement.

2.7 Existing applications

Application	What it does	Gap for our users
Google Lens	Recognises and translates text in photos, can read aloud	General-purpose, many menus, assumes the user can read and navigate the result
Microsoft Seeing AI	Reads text aloud for blind users	Mainly English; limited Hindi support
Envision AI	Reads text for blind and low-vision users	Paid subscription; not designed for Hindi medicine labels
Tesseract-based apps	Offline OCR with a Hindi model	Weak on camera photos, matras and conjuncts; no speech

2.8 Gap analysis

No existing tool combines all of the following:

1. A recogniser trained for Hindi medicine, prescription and notice text, including doses, prices and dates
2. Mixed Hindi–English speech that switches voice by script
3. A Hindi-first, icon-led interface with automatic read-aloud for low-literacy users
4. Built-in, privacy-safe impact measurement for an NGO field pilot
5. A model small enough to move onto the phone later

GramText is designed to fill this gap.

CHAPTER 3

Requirements analysis

3.1 Feasibility study

Technical feasibility. All components use mature open-source tools: PyTorch and timm for training, ONNX Runtime for inference, FastAPI for the server, and Kotlin with Jetpack Compose and CameraX for the app. Training fits on a free Google Colab T4 GPU. The trained model is small (about 0.85M parameters), so it runs on a CPU-only server. The project is technically feasible.

Economic feasibility. The project uses no paid software. Training uses free Colab, hosting uses the Render free plan (512 MB RAM), and gTTS needs no API key. The Gemini backup has a free usage tier. Users need only a basic Android phone (Android 8.0 or later) and a mobile data connection. The project is economically feasible.

Operational feasibility. The app needs no reading skill to operate: the user takes a photo and hears the result. Hindi labels and icons guide every step. The NGO partner can run a field pilot with its own health workers, and the metrics endpoint reports usage without extra effort. The project is operationally feasible.

Legal and ethical feasibility. The app stores no photos or scanned text. Feedback corrections are sent only when the user chooses to send them. Every screen states that the app reads text only and gives no medical advice.

3.2 Stakeholders

Stakeholder	Role	Needs
Low-literacy rural users	Primary users	Hear medicine labels and notices read aloud, simply and quickly
NGO health and literacy workers	Field partners	Introduce the app, collect feedback, report outcomes
Project team	Developers	Build, train, deploy and evaluate the system
Project guide and college	Evaluators	A working system with documented design and results

3.3 Functional requirements

ID	Requirement
FR-01	Capture a label with the camera, or choose a photo from the gallery
FR-02	Validate the image type and size, and fix its rotation
FR-03	Detect and recognise printed Devanagari and English text
FR-04	Preserve matras and conjuncts in the recognised text
FR-05	Show the recognised text and allow the user to edit it
FR-06	Read the text aloud in Hindi and English, including mixed labels
FR-07	Play slowly on request, and replay without using the network
FR-08	Collect helpful/not helpful feedback, with optional corrections, for the field pilot
FR-09	Report anonymous impact metrics
FR-10	Show clear errors in the user's language, and never give medical advice

3.4 Non-functional requirements

ID	Category	Requirement
NFR-01	Performance	Speech should start within a few seconds of taking the photo on a normal 4G connection
NFR-02	Low bandwidth	Photos are shrunk on the phone to at most 2048 px (about 400 KB) before upload
NFR-03	Resource use	The server must run within 512 MB RAM (Render free plan)
NFR-04	Usability	Hindi-first labels, large icon buttons, 24 sp text, automatic read-aloud, TalkBack labels
NFR-05	Privacy	No photos or scanned text stored; only anonymous counts and voluntary corrections
NFR-06	Reliability	If our model is not confident, fall back to Gemini; if Gemini fails, return our model's result
NFR-07	Security	Validate every upload as a real JPG/PNG/WEBP under 10 MB; keep API keys out of source control
NFR-08	Maintainability	Modular code: separate services for detection, recognition, TTS and metrics
NFR-09	Portability	The model is exported to ONNX so the same file can later run on a phone

3.5 Hardware requirements

Component	Minimum
User's phone	Android 8.0 (API 26) or later, rear camera, 2 GB RAM, mobile data or Wi-Fi
Server	1 shared CPU, 512 MB RAM (Render free plan)
Training	NVIDIA T4 GPU, 16 GB (Google Colab free tier)
Development PC	8 GB RAM, 64-bit Windows, macOS or Linux

3.6 Software requirements

Layer	Software
Mobile app	Kotlin, Jetpack Compose (Material 3), CameraX, OkHttp 4.12, Kotlin coroutines; compile and target SDK 36, minimum SDK 26
Backend	Python 3, FastAPI, Uvicorn, Pillow, NumPy, ONNX Runtime, onnxtr (docTR DBNet), gTTS, google-genai, SQLite
Machine learning	PyTorch 2.4+, timm 1.0+, ONNX 1.16+, uHarfBuzz, freetype-py
Web demo (optional)	React 19, Vite 8, lucide-react
Tools	Android Studio, VS Code, Git and GitHub, Google Colab, Render

CHAPTER 4

System design

4.1 System architecture

GramText follows a three-tier client-server architecture. The Android app is the presentation tier. The FastAPI backend is the application tier, holding the detection, recognition, speech and metrics services. A SQLite database stores only anonymous counts and voluntary feedback. Two external services support the backend: Google Gemini as the backup recogniser, and the Google Translate speech service used by gTTS.

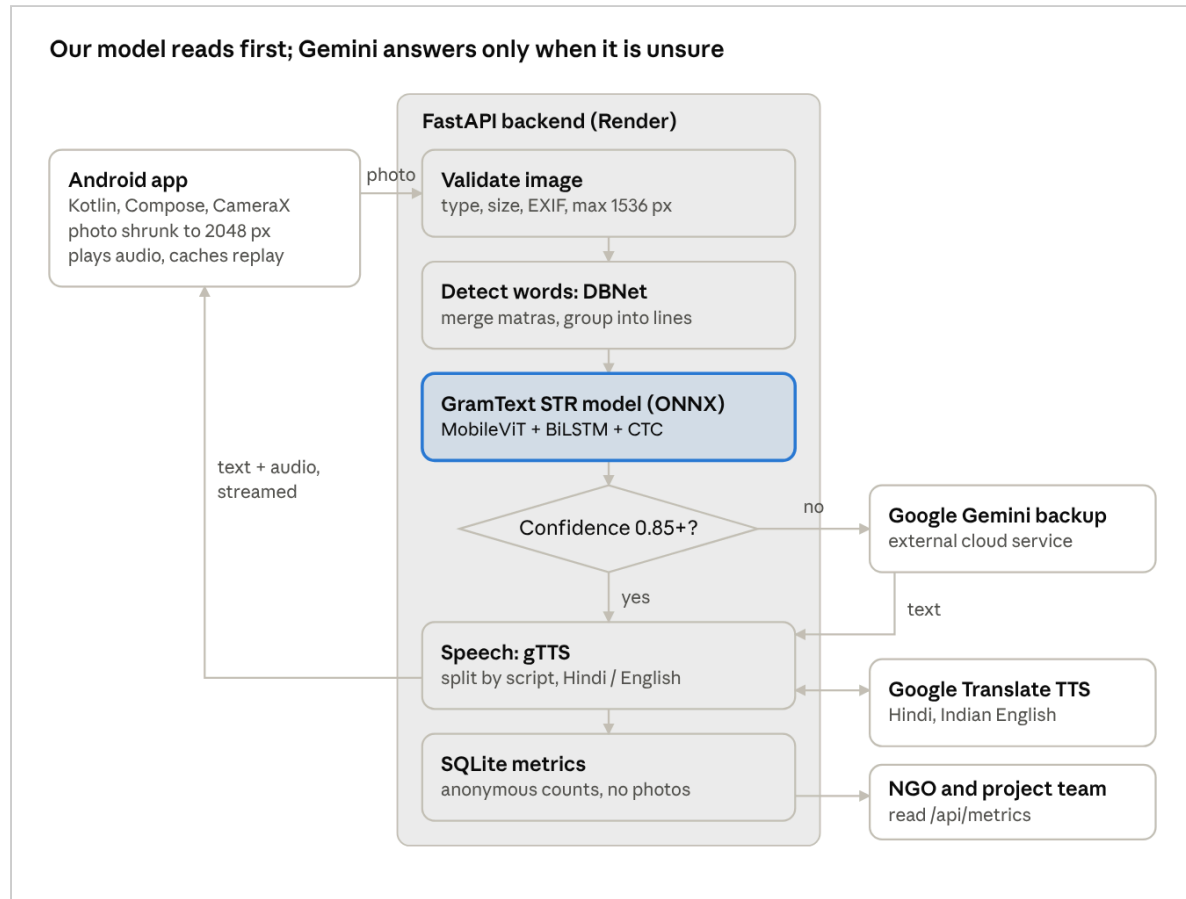


Figure 4.1: GramText system architecture

The app talks only to the backend. The backend calls Gemini only when our model is missing or unsure, and it stores counts, never photos.

The main flow for one scan is:

1. The app captures a photo, fixes its rotation and shrinks it to at most 2048 px.
2. It uploads the JPEG to the server over HTTPS.
3. The server validates the image and shrinks it further to at most 1536 px.
4. DBNet finds the words and groups them into lines.
5. Our STR model reads every word crop in one batch and reports a confidence score.
6. If the confidence is 0.85 or higher, the server uses our result. Otherwise it asks Gemini.
7. The text is split into sentences, and each sentence is sent to gTTS as soon as it is ready.
8. Text and audio are streamed back to the app, which shows the text and plays the speech.

4.2 Module design

Module	Location	Responsibility
Camera and capture	android/.../ui/CameraScreen.kt, LiveAnalyzer.kt	Camera preview, scan box, steady-frame detection, gallery picker
Image preparation	android/.../data/ImagePrep.kt	Fix EXIF rotation, crop to the scan box, shrink to at most 2048 px, encode JPEG
API client	android/.../data/ApiClient.kt	HTTP calls, streamed response parsing, anonymous client ID header
Speech player	android/.../audio/SpeechPlayer.kt	Plays streamed MP3 chunks in order, caches audio for offline replay, slow-speed playback
Result screen	android/.../ui/ResultScreen.kt	Editable text, read-aloud button, feedback and correction
Settings	android/.../ui/SettingsScreen.kt, Prefs.kt	Server address, speech speed, technical details toggle
API layer	backend/app/main.py	Endpoints, upload checks, error handling, CORS
Image validation	backend/app/utils/validation.py	Real-image check, EXIF rotation, resize
Text detector	backend/app/services/detector.py	DBNet word boxes, merge stray matra fragments, group into lines
STR model	backend/app/services/str_model.py	Load ONNX model, preprocess crops, CTC decode, confidence
OCR pipeline	backend/app/services/ocr_pipeline.py	Choose between our model and Gemini

Module	Location	Responsibility
Gemini backup	backend/app/services/gemini_ocr.py	Streamed OCR with model fallback and retry
Speech	backend/app/services/gtts_tts.py	Split text by script, Hindi or English voice, MP3 output
Read-aloud streamer	backend/app/services/reader.py	Overlap OCR and TTS, stream text and audio events
Metrics store	backend/app/db.py	SQLite tables and the impact metrics query

4.3 Data flow

Level 0 (context). The user gives a photo to GramText and receives text and speech. GramText exchanges images and text with the Gemini service, and text and audio with the speech service. The NGO reads aggregated metrics.

Level 1. The system has five processes:

1. **Capture and prepare** turns a camera frame into a small JPEG.
2. **Validate** checks the file and fixes its orientation.
3. **Recognise** runs detection and our STR model, or the Gemini backup.
4. **Speak** splits the text by script and produces MP3 audio.
5. **Record** writes anonymous scan, speech and feedback rows to the metrics store, from which the metrics report is built.

4.4 Use cases

Use case	Actor	Description
UC-1 Scan a label	User	Hold the label in the scan box or tap the camera button; hear it read aloud
UC-2 Pick from gallery	User	Choose an existing photo and hear it read aloud
UC-3 Replay or slow down	User	Replay the cached audio, or play it at slow speed
UC-4 Correct and give feedback	User	Mark the result helpful or not, optionally fix the text and send it
UC-5 Change server	User or NGO worker	Set the server address in Settings
UC-6 View impact metrics	NGO or project team	Open the metrics endpoint to see usage and accuracy

4.5 Sequence of a scan

The app calls a single endpoint, POST /api/read, and the server answers with a stream of JSON lines (NDJSON). This lets speech start before recognition is fully finished:

1. App to server: multipart JPEG with an anonymous X-Client-Id header.
2. Server to app: a **text** event with the text read so far.
3. Server to app: one **audio** event per sentence, each with base64 MP3 data, in order.
4. Server to app: a **done** event with the language, engine, confidence, scan ID and timings.
5. If anything fails: an **error** event with a friendly message and the stage (ocr or tts).

Up to six sentences are sent to gTTS in parallel, but the audio is always returned in reading order.

4.6 API design

Endpoint	Input	Output
POST /api/read	Multipart image	Streamed NDJSON events: text, audio, done, error
POST /api/ocr	Multipart image	text, language, engine (str or gemini), confidence, scan_id
POST /api/tts	JSON {text} (at most 1,500 characters)	MP3 audio
POST /api/feedback	JSON {scan_id, helpful, corrected_text?, comment?}	feedback_id
GET /api/metrics	None	Impact metrics and model information
GET /api/health	None	Which engines are ready

4.7 Database design

The database holds three tables. None stores an image or the scanned text. The only text stored is a correction the user chooses to send.

scans

Column	Type	Meaning
id	INTEGER, primary key	Scan ID
ts	REAL	Time of the scan
engine	TEXT	str or gemini

Column	Type	Meaning
confidence	REAL	Our model's confidence
latency_ms	INTEGER	Time taken
chars	INTEGER	Length of the text
language	TEXT	hi, en or und
success	INTEGER	1 if text was found
client	TEXT	Anonymous random device ID

tts

Column	Type	Meaning
id	INTEGER, primary key	Request ID
ts	REAL	Time
chars	INTEGER	Length of the text spoken
latency_ms	INTEGER	Time taken
success	INTEGER	1 if audio was produced

feedback

Column	Type	Meaning
id	INTEGER, primary key	Feedback ID
ts	REAL	Time
scan_id	INTEGER	The scan it refers to
helpful	INTEGER	1 = helpful, 0 = not helpful
corrected_text	TEXT	Optional correction from the user
comment	TEXT	Optional comment
client	TEXT	Anonymous random device ID

4.8 User interface design

The interface is designed for people who may not read well:

- **Hindi first.** Every label is in Hindi, for example फ़ोटो खींचें (take photo) and पढ़कर सुनाएँ (read aloud).
- **Icons with words.** Large buttons pair an icon with a short word, so the meaning is clear without reading.

- **Automatic speech.** The text is read aloud as soon as it is recognised; the user does not need to find a button.
- **Large text.** Recognised text is shown at 24 sp and can be edited.
- **Accessibility.** All buttons have TalkBack labels for screen-reader users.
- **Safety.** A disclaimer on every screen states that the app gives no medical advice.

CHAPTER 5

The GramText STR model

Our recogniser reads a single word image and outputs its text. It is a MobileViTv2-0.5 backbone followed by a two-layer BiLSTM and a CTC output layer over 213 characters. It has about 0.85 million parameters. This chapter covers the character set, the training data, the architecture, training, export and evaluation.

5.1 Character set

The model recognises text at the level of Unicode code points. Conjuncts and matras are therefore learned as sequences of code points, exactly as Unicode stores them.

Group	Characters	Count
Devanagari	Every assigned code point in U+0900 to U+097F	128
Latin letters and digits	A–Z, a–z, 0–9	62
Punctuation and symbols	Space . , ; ! ? ' " () [] - / % & + * # @ ₹ –	23
Total		213 , plus 1 CTC blank = 214 output classes

Before encoding, every label is normalised to Unicode NFC form. Curly quotes are replaced with straight quotes, the em dash with an en dash, and zero-width joiners are removed. Repeated spaces are collapsed.

5.2 Synthetic training data

Real, labelled photos of Hindi medicine labels do not exist in large numbers. We therefore generate training images on the fly. Each request to the dataset renders a new random sample, so the training set is effectively unlimited.

Fonts. 14 Google Fonts covering Devanagari and Latin in varied styles: Hind (regular and bold), Mukta (regular and bold), Poppins (regular and semibold), Martel, Tiro Devanagari Hindi, Rozha One, Noto Sans Devanagari, Kalam, Amita, Teko and Laila.

Text shaping. Each word is shaped with HarfBuzz and rasterised with FreeType. HarfBuzz applies the OpenType rules that reorder the short-i matra and form conjuncts. This makes the rendered word look exactly like printed text.

Vocabulary.

- 20,000 frequent Hindi words and 47,000 frequent English words from the FrequencyWords corpus
- A domain list of medicine, prescription and notice words, repeated 20 times so the model sees them often. Examples: दवा (medicine), खुराक (dose), भोजन के बाद (after food), समाप्ति तिथि (expiry date), योजना (scheme), paracetamol, MRP, mg
- Generated tokens for doses (500mg, 5 ml), prices (₹120), dates (12/05/2027) and times (8:30)

Sample mix.

Sample type	Share
Hindi, 1 to 3 words	45%
English, 1 to 2 words, in lower, Capitalised or UPPER case	35%
A number token, sometimes after a word	12%
Mixed Hindi + English	8%

Labels are cut to at most 25 characters, and 10% end with a punctuation mark such as । (danda).

Augmentation. Each image is rendered at a random font size of 28 to 64 px with random padding. The following effects are applied at random to imitate phone photos:

Effect	Probability
Random background and ink tone, with a lighting gradient and noise	Always
Light text on a dark background (signboards)	20%
Rotation of up to ± 4 degrees	50%
Shear (italic-like slant)	30%
Gaussian blur	40%
Downscale and upscale (low resolution)	40%
JPEG compression at quality 20 to 80	40%

Real word crops (from IIIT-ILST, IndicSTR12 or the field pilot) can be mixed in, making up about 20% of training samples.

5.3 Preprocessing

Every word crop is converted to grayscale and resized to a height of 32 px, keeping its aspect ratio, with a maximum width of 256 px. It is then padded on the right to 256 px using the median value of its last column. Pixel values are scaled to the range -1 to 1 . The server uses exactly the same steps, so training and inference match.

5.4 Architecture

The model has three parts:

1. **Backbone: MobileViTv2-0.5** (from the timm library), taking a 1-channel image and initialised with ImageNet weights. We keep the output of its third stage. The stride of the first MobileViT block is changed from (2, 2) to (2, 1), so the network halves the height but not the width. A 32×256 input then gives a 4×64 feature map, that is, 64 time steps. This is enough for long Devanagari words at code-point level.
2. **Sequence model: two-layer bidirectional LSTM** with 128 hidden units in each direction and dropout 0.1. The feature map is averaged over its height, giving a sequence of 64 feature vectors. The BiLSTM reads this sequence in both directions, so each step sees the context of the whole word.
3. **Output: a linear layer** maps each of the 64 steps to scores for the 214 classes.

Stage	Output shape
Input image	$B \times 1 \times 32 \times 256$
MobileViTv2-0.5, stages 0–2 (modified stride)	$B \times C \times 4 \times 64$
Mean over height, transpose	$B \times 64 \times C$
BiLSTM, 2 layers, 128 units each way	$B \times 64 \times 256$
Linear layer	$B \times 64 \times 214$

5.5 CTC loss and decoding

For an image x with label y , CTC sums the probability of every alignment π of length $T = 64$ that collapses to y . Collapsing merges repeated symbols and then removes blanks. The training loss is the negative log of this sum:

$$L_{\text{CTC}} = -\log \sum_{\pi \in B^{-1}(y)} \prod_{t=1}^T p_t(\pi_t | x) \quad (5.1)$$

Here B is the collapsing function and p_t is the softmax output at step t .

At inference, greedy decoding takes the most likely class at each step, merges repeats and removes blanks. The **confidence** of a word is the mean probability of the characters that were emitted. For a whole photo, the server weights each word's confidence by its length. A photo whose overall confidence is below 0.85 is sent to the Gemini backup.

5.6 Training

Setting	Value
Hardware	Google Colab, NVIDIA T4 GPU
Steps	40,000 (the notebook setting; about 3–5 hours on a T4)
Batch size	128
Optimiser	AdamW, weight decay 0.01
Learning rate	One-cycle schedule, peak 1×10^{-3} , 5% warm-up
Precision	Mixed (float16) with gradient scaling
Loss	CTC, blank = 0, infinite losses zeroed
Validation	Every 2,000 steps on 1,000 held-out samples; the best checkpoint is kept
Resilience	Checkpoints are saved to Google Drive; training resumes after a Colab disconnect

The notebook `ml/train_colab.ipynb` runs the whole process: install packages, prepare data, train, evaluate and export.

5.7 Export

The best checkpoint is exported to ONNX (opset 17) with a dynamic batch size. The character set and training metadata are stored inside the ONNX file, so a single file is all the server needs. The export script checks that the ONNX output matches the PyTorch output to within 0.001 for several batch sizes. A TorchScript export is also available. The server prefers the ONNX file, because ONNX Runtime needs far less memory than PyTorch and fits the 512 MB free hosting plan.

5.8 Evaluation metrics

The evaluation script reports two metrics, overall and separately for Hindi and English samples:

$$\text{Character accuracy} = 1 - \frac{\sum \text{EditDistance}(\hat{y}, y)}{\sum |y|} \quad (5.2)$$

Word accuracy is the share of samples whose prediction matches the label exactly. Edit distance is the Levenshtein distance: the minimum number of insertions, deletions and substitutions that turn the prediction into the label. The script writes a JSON report with the metrics and up to 50 example errors.

CHAPTER 6

Implementation

The system is implemented in about 1,960 lines of Kotlin for the app, a Python backend organised into separate services, and about 710 lines of Python for the machine-learning pipeline. The project folder is organised as follows:

Folder	Contents
android/	Kotlin + Jetpack Compose + CameraX app
backend/	FastAPI server: detector, our STR model, Gemini backup, gTTS, metrics
ml/	PyTorch training: synthetic data, MobileViT-CTC model, Colab notebook, evaluation, export
frontend/	Optional React web demo of the same API
docs/	Project notes, NGO letter, feedback form and impact report templates

6.1 Backend

The backend is a FastAPI application served by Uvicorn. On start-up it creates the SQLite tables and, in background threads, loads the detector and STR model and opens keep-alive connections to the speech and Gemini services. The first real scan is therefore fast.

Endpoint functions are written as plain (non-async) functions. FastAPI runs them in a worker thread pool, so a slow model or network call does not block other users.

Upload validation. The server accepts only JPG, PNG and WEBP files under 10 MB. It reads at most 10 MB + 1 byte, so an oversized file is rejected without reading all of it. Pillow then opens the bytes to confirm they are a real image. This also guards against decompression bombs. The image is rotated using its EXIF tag and shrunk to at most 1536 px on its longer side.

Text detection. The detector uses the DBNet model with a MobileNetV3-Large backbone from onnxtr, the ONNX build of docTR. It needs no PyTorch. Two extra steps are specific to Devanagari:

- DBNet sometimes detects a matra above the headline, such as the sign in ँ or ो, as a separate tiny box. Boxes shorter than 60% of the median word height are merged into the word they overlap.

- Word boxes are grouped into lines from top to bottom and sorted left to right, so the text reads in the right order.

Recognition. All word crops from the photo are sent to the STR model in batches of 32. The ONNX session uses two CPU threads. The output is decoded greedily, and each word's confidence is kept.

Engine choice. The OCR pipeline supports three modes, set by the OCR_ENGINE variable:

Mode	Behaviour
auto (default)	Our model first; Gemini if our model is missing or its confidence is below 0.85
str	Our model only, for evaluation and the viva demonstration
gemini	Gemini only

In auto mode, if Gemini is unavailable and our model produced some text, the server returns our result anyway. A low-confidence answer is better than no answer.

Gemini backup. The backup uses the gemini-flash-lite model with low "thinking" effort. In our tests this took about 2 seconds per scan, compared with about 17 seconds for the default Flash setting. Gemini returns text in pieces as it reads, and the server passes each piece on immediately. If a model is retired or overloaded, the server tries the next model in a fallback list, and retries a busy model once.

Speech. The speech service splits the text into runs by script. Devanagari letters start a Hindi run and Latin letters start an English run. Digits, spaces and punctuation stay with the current run, so "दवा 2 बार" stays one Hindi run and "Paracetamol 500mg" stays one English run. Hindi runs use the Hindi voice. English runs use the Indian-English voice (the co.in domain). The MP3 audio from each run is joined into one file. Each worker thread keeps one HTTPS connection open, which saves about 0.2–0.4 seconds per request.

Streaming read-aloud. The /api/read endpoint overlaps recognition and speech. As soon as a full sentence is recognised (ending in a full stop, danda, question mark or a new line), it is sent to gTTS. Long text without punctuation is cut at a space every 100 characters. Six speech workers run in parallel, and the audio events are returned in reading order. The response reports the OCR time and the time to the first audio.

Metrics. Every scan, speech request and feedback item is written to SQLite. The /api/metrics endpoint returns totals, success counts, average latency, the split by engine and language, our model's average confidence, the helpful rate, the number of corrections, unique devices and scans per day for the last 30 days.

Error handling. Every service raises a `ServiceError` with a friendly message and an HTTP status code. For example, a rate limit from the speech service becomes "The free API limit has been reached. Please try again later." (HTTP 429). The app shows the message in the user's language.

6.2 Android app

The app is written in Kotlin with Jetpack Compose (Material 3). It follows the MVVM pattern: a single `GramViewModel` holds the screen state as a Kotlin `StateFlow`, and the screens draw that state. Network calls run in coroutines. The app supports Android 8.0 (API 26) and later.

Screen or class	What it does
CameraScreen	CameraX preview with a scan box, a large capture button, a gallery button, and live mode
LiveAnalyzer	In live mode, checks the scan box every 0.2 s and sends a frame only when a new label is held steady
ImagePrep	Reads the photo, fixes rotation, crops to the scan box, shrinks to at most 2048 px, saves as JPEG at quality 85
ApiClient	OkHttp client; parses the streamed NDJSON events; sends an anonymous random client ID
SpeechPlayer	Queues MP3 chunks and plays them in order; caches the full audio so replay needs no network
ResultScreen	Shows the text at 24 sp, editable; read-aloud button; helpful/not helpful buttons and correction
SettingsScreen	Server address, app language, auto read-aloud, slow speech, show technical details
Strings	All interface text in Hindi and English

Live mode. The `LiveAnalyzer` makes a 32×24 grayscale "signature" of the scan box every 0.2 seconds. A frame is sent only when the box content is steady (the camera is not moving) and different from the last frame sent (a new label). It never sends a frame while the previous one is still being read, and never more than once every 0.6 seconds. Text outside the scan box is ignored. A user can therefore move from one medicine strip to the next and hear each one without pressing a button.

Slow speech. When slow speech is on, audio plays at 0.8 times normal speed.

Privacy. The client ID is a random value generated on the phone. It is used only to count unique devices and cannot identify a person.

6.3 Web demo

An optional web demo built with React 19 and Vite offers the same camera, result and settings screens in a browser. It calls the same API, so the system can be shown on a laptop or on any phone browser without installing the APK.

6.4 Deployment

The backend is deployed on Render using a blueprint file (render.yaml):

- The free web service plan with 512 MB RAM. It sleeps after 15 minutes of inactivity, so the first request after that takes about 50 seconds.
- The build step installs the dependencies and pre-downloads the DBNet weights, so the first scan after a deploy is fast.
- The Gemini API key is entered as a secret in the Render dashboard and is never stored in Git.
- Render checks `/api/health` to confirm the service is running.

The ONNX version of our model and the detector together use about 280 MB of memory, which fits the free plan. Vercel was considered and rejected: its serverless functions cannot keep the warm connection pool that gives fast speech, and PyTorch exceeds its size limit.

The Android APK is built with Gradle. The server address can be fixed at build time with `-PapiBaseUrl`, or changed later in the app's Settings without rebuilding.

6.5 Security measures

Risk	Measure
Malicious or oversized uploads	Type and size checks; Pillow opens the file to confirm it is a real image; decompression-bomb guard
Leaked API keys	Keys only in the <code>.env</code> file (ignored by Git) or the hosting dashboard
Cross-site requests	CORS allows only the web demo's own addresses and local-network addresses
Privacy of users	No photos or scanned text stored; anonymous random device ID; corrections only with consent
Misuse as medical advice	Disclaimer on every screen; the app only reads text

CHAPTER 7

Testing

All 14 automated backend checks passed on 4 October 2026. Device and field tests are listed with their expected results; their outcomes are to be recorded during the demo and the field pilot.

7.1 Testing strategy

Level	What is tested	How
Unit	Script splitting, sentence segmentation, image validation	Python checks against the service functions
Integration	API endpoints with validation and error handling	FastAPI TestClient against the real application
Model	Learning ability and accuracy	CPU overfit test; eval.py on held-out synthetic and real crops
System	Full scan from phone to speech	Android emulator and a real phone against the backend
User acceptance	Usefulness for low-literacy users	NGO field pilot with the feedback form and in-app feedback

7.2 Automated backend tests

These checks were run against the backend code with the FastAPI TestClient. In this run no trained model file was installed, so the health check reported `str_model_ready = false` and `gemini_ready = true`.

ID	Test	Input	Expected	Result
TC-01	Split mixed-script text	"दवा 2 बार Paracetamol 500mg"	Hindi run "दवा 2 बार", then English run "Paracetamol 500mg"	Pass
TC-02	Split Hindi-only text	"भोजन के बाद लें"	One Hindi run	Pass
TC-03	Cut sentences at the danda	"पहला वाक्य। दूसरा वाक्य"	First sentence returned; the rest held back	Pass
TC-04	Cut long text without punctuation	160 characters, no full stop	Every segment at most 100 characters	Pass

ID	Test	Input	Expected	Result
TC-05	Reject a fake image	Bytes that are not an image	Error 400	Pass
TC-06	Shrink a large photo	4000 × 3000 JPEG	Longer side 1536 px	Pass
TC-07	Health check	GET /api/health	status "healthy"	Pass
TC-08	Reject the wrong file type	A text file	Error 400	Pass
TC-09	Reject an empty upload	0 bytes	Error 400	Pass
TC-10	Reject an oversized upload	10 MB + 5 bytes	Error 413	Pass
TC-11	Reject empty speech text	Spaces only	Error 400	Pass
TC-12	Reject over-long speech text	1,501 characters	Error 400	Pass
TC-13	Record feedback	helpful = true	success = true and a feedback ID	Pass
TC-14	Metrics report	GET /api/metrics	Contains scans, tts, feedback and unique_devices	Pass

7.3 Model tests

ID	Test	Expected	Result
MT-01	CPU overfit on 64 fixed samples, 1,000 steps	Loss falls and accuracy rises, showing the model and loss are wired correctly	Character accuracy reached 87%
MT-02	ONNX export matches PyTorch	Outputs equal within 0.001 for several batch sizes	Built into export.py; checked at every export
MT-03	Accuracy on held-out synthetic data (2,000 samples)	Character and word accuracy, overall, Hindi and English	[Record from eval_synthetic.json after Colab training]
MT-04	Accuracy on real word crops	Character and word accuracy on real photos	[Record from eval_real.json]

7.4 System and device tests

ID	Scenario	Expected result	Result
ST-01	Open the app with the server running	Home screen shows "सर्वर जुड़ा है" (server connected)	[]

ID	Scenario	Expected result	Result
ST-02	Photograph a Hindi medicine label	Text appears and is read aloud in the Hindi voice	[]
ST-03	Photograph a mixed Hindi–English label	Hindi parts in the Hindi voice, English parts in the English voice	[]
ST-04	Choose a photo from the gallery	Same as a camera photo	[]
ST-05	Live mode: hold two labels in turn	Each label is read once, without pressing a button	[]
ST-06	Tap read aloud again with mobile data off	Cached audio plays	[]
ST-07	Turn on slow speech	Audio plays at 0.8× speed	[]
ST-08	Edit the text, tap 🗨️, send the correction	Feedback recorded; metrics show one more correction	[]
ST-09	Photo with no text	Friendly "no readable text" message in Hindi	[]
ST-10	Server not reachable	Friendly connection error in Hindi	[]
ST-11	TalkBack on	Every button is announced with its name	[]
ST-12	First request after the Render service sleeps	Response within about 50 seconds; later requests fast	[]

7.5 User acceptance testing

User acceptance testing is planned with the partner NGO using the community feedback form in docs/templates. Participants scan their own medicine labels and notices with the app, then answer simple spoken questions: whether the reading was correct, whether they understood it, and whether they would use the app again. The in-app helpful/not helpful buttons and the metrics endpoint provide the matching usage data. Results are to be recorded in the impact report (Chapter 8).

CHAPTER 8

Results and SDG impact

The system works end to end: in development testing on 3 October 2026, every speech request succeeded and a scan took about 3.3 seconds on average. All successful scans in that session were read by the Gemini backup, because the trained STR model had not yet been installed on the server.

8.1 Delivered outputs

Output	Status
Android app (Kotlin, Compose, CameraX) with camera, gallery and live modes	Built; APK produced (GramText.apk)
FastAPI backend with detection, recognition, speech, feedback and metrics	Built; deployable on Render
GramText STR model code: synthetic data, MobileViT-CTC model, training notebook, evaluation, ONNX export	Built; CPU overfit test passed; full Colab training run pending
Gemini backup recogniser	Working
Hindi and English speech, including mixed labels	Working
Optional React web demo	Built
NGO collaboration letter, community feedback form, impact report	Templates ready; to be completed with the NGO

8.2 Development testing results

These figures come from the backend's metrics store after one day of development testing on 3 October 2026, across 6 test devices. They describe the developers' own testing, not the field pilot.

Metric	Value
Scan requests	90
Scans that returned text	45 (50%)
Average scan time, all requests	3.26 s
Successful scans by language	32 Hindi, 13 English
Average scan time, Hindi / English	3.33 s / 4.46 s

Metric	Value
Engine for successful scans	Gemini 45, our STR model 0 (model not yet installed)
Speech requests	44, all successful
Average speech time	3.42 s
Feedback items	1 (marked helpful)

Many of the scans without text came from live mode and test frames that held no readable text, so the 50% figure should not be read as recognition accuracy. Accuracy will be measured with eval.py on labelled test sets (Chapter 7, MT-03 and MT-04) and through user feedback in the field pilot.

8.3 Model results

In a CPU overfit test on 64 fixed samples, character accuracy reached 87% within 1,000 steps. This shows that the data pipeline, model and CTC loss work together correctly. Full training on Colab (40,000 steps) and evaluation on held-out synthetic and real crops are the next steps. Their results go into the table below.

Test set	Character accuracy	Word accuracy
Synthetic held-out, Hindi	[]	[]
Synthetic held-out, English	[]	[]
Real crops (IIIT-ILST or field pilot)	[]	[]

8.4 Screenshots

Insert the following screenshots from the app:

1. Home screen showing "सर्वर जुड़ा है" (server connected)
2. Camera screen with the scan box
3. Result screen with recognised Hindi text and the read-aloud button
4. Result screen for a mixed Hindi–English label, with technical details showing the engine and confidence
5. Feedback and correction dialog
6. Settings screen
7. The /api/metrics output in a browser

8.5 SDG alignment

SDG	Target addressed	How GramText contributes	How it is measured
SDG 3: Good Health and Well-being	Safe use of medicines	Users hear dosage instructions, warnings and expiry dates on medicine labels	Scans of medicine labels; helpful rate; feedback form answers on medicine use
SDG 4: Quality Education	Adult literacy and digital skills	Turns printed text into speech; editable text lets users connect spoken and written words	Unique devices; repeat use; feedback form answers on confidence with text
SDG 10: Reduced Inequalities	Equal access to information and services	Gives low-literacy users independent access to government scheme notices	Scans of notices; share of Hindi scans; participants who cannot read Hindi

8.6 NGO engagement and field pilot plan

- 1. Agreement.** Sign the NGO collaboration letter (docs/templates/ngo_collaboration_letter.md) with [NGO name].
- 2. Preparation.** Install the APK on volunteers' phones or the NGO's devices, and set the server address in Settings.
- 3. Briefing.** NGO health workers explain the app to participants and stress that it reads text only and gives no medical advice.
- 4. Pilot.** Participants scan their own medicine labels and scheme notices over [number] weeks.
- 5. Feedback.** Health workers fill in the community feedback form with each participant. In-app feedback and corrections are collected automatically.
- 6. Report.** The team fills in the impact report template with metrics from /api/metrics, model accuracy from eval.py, and the feedback form averages.
- 7. Improvement.** Corrections submitted during the pilot become real training data for the next version of the model.

CHAPTER 9

Limitations, future work and conclusion

9.1 Deviations from the approved proposal

The proposal's stack was Python, PyTorch, a lightweight MobileViT STR model, TFLite, Kotlin or Flutter, an offline-first architecture, gTTS and an NGO partnership. This phase follows it, with the differences below.

Proposal	This phase	Reason
Offline-first architecture	Online client-server	The team deferred offline work to Phase 2 to deliver a complete, measurable system first
TFLite on the phone	ONNX on the server	ONNX Runtime fits the 512 MB free server; the same small model can be converted for the phone later
Kotlin/Flutter	Kotlin with Jetpack Compose	Native camera control with CameraX and a single code base to maintain
(not in proposal)	Google Gemini as a backup recogniser	Keeps the app useful while our model is trained and improved
(not in proposal)	DBNet text detector (docTR)	Our STR model reads single words, so words must be found first
(not in proposal)	React web demo	Lets the system be shown in a browser without installing the APK

9.2 Limitations

- **Needs a network.** Recognition and speech both run online, so the app needs mobile data or Wi-Fi. This is the main gap against the offline-first objective.
- **Model not yet deployed.** Until the trained ONNX file is placed in backend/models, every scan is read by Gemini, an external cloud service.
- **Mostly synthetic training data.** Accuracy on real photos is expected to be lower than on synthetic test data until real crops and pilot corrections are added.
- **Printed text only.** Handwritten prescriptions are not supported.
- **Speech service.** gTTS relies on Google Translate's public speech service, which has no service-level agreement and may rate-limit requests.

- **Free hosting.** The Render free plan sleeps after 15 minutes idle, so the first request then takes about 50 seconds. Its disk is temporary, so metrics reset on every deploy or restart.
- **Privacy of the backup path.** When Gemini is used, the photo is sent to Google. No photos are stored by our server.
- **Field pilot pending.** Impact figures in this report come from development testing. Real impact figures depend on the NGO pilot.

9.3 Future work

1. **Offline-first (Phase 2).** Convert the 0.85M-parameter model to TFLite or run the ONNX file on the phone with ONNX Runtime Mobile. Add an on-device detector and an offline Hindi voice, so the whole app works without a network.
2. **Train and deploy our model.** Complete the 40,000-step Colab run, evaluate it, and install the ONNX file on the server so most scans use our own model.
3. **More real data.** Add IIIT-ILST and IndicSTR12 crops and the corrections collected in the pilot, then retrain.
4. **Better detection.** Fine-tune the detector on Devanagari labels, and add automatic cropping and perspective correction for curved packaging.
5. **More languages.** Extend to Marathi (also Devanagari), then Bengali, Gujarati, Punjabi, Tamil and Telugu.
6. **Durable metrics.** Move the metrics store to a hosted database so figures survive server restarts.
7. **Handwriting.** Explore recognition of handwritten prescriptions as a separate research task.

9.4 Conclusion

GramText shows that a small, purpose-built model and a simple, Hindi-first interface can turn a phone camera into a reading aid for people who struggle with printed text. The project delivered a working Android app, a deployable FastAPI backend, a complete training pipeline for a 0.85M-parameter MobileViT-CTC recogniser, mixed Hindi–English speech, and built-in, privacy-safe impact measurement.

The main technical contributions are the code-point-level Devanagari recogniser trained on HarfBuzz-shaped synthetic data, the matra-fragment merging step after detection, and the streamed pipeline that starts speech before recognition finishes. The main remaining work is

to deploy the trained model, move inference onto the phone for offline use, and complete the field pilot with the NGO. With those steps, GramText can make a direct contribution to SDG 3, SDG 4 and SDG 10 in rural communities.

References

1. R. Smith, "An Overview of the Tesseract OCR Engine," in Proc. ICDAR, 2007.
2. X. Zhou et al., "EAST: An Efficient and Accurate Scene Text Detector," in Proc. CVPR, 2017. arXiv:1704.03155
3. Y. Baek et al., "Character Region Awareness for Text Detection," in Proc. CVPR, 2019. arXiv:1904.01941
4. M. Liao, Z. Wan, C. Yao, K. Chen and X. Bai, "Real-time Scene Text Detection with Differentiable Binarization," in Proc. AAAI, 2020. arXiv:1911.08947
5. Mindee, "docTR: Document Text Recognition," open-source library. github.com/mindee/doctr
6. B. Shi, X. Bai and C. Yao, "An End-to-End Trainable Neural Network for Image-based Sequence Recognition and Its Application to Scene Text Recognition," IEEE TPAMI, 2017. arXiv:1507.05717
7. A. Graves, S. Fernández, F. Gomez and J. Schmidhuber, "Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks," in Proc. ICML, 2006.
8. B. Shi et al., "ASTER: An Attentional Scene Text Recognizer with Flexible Rectification," IEEE TPAMI, 2018.
9. D. Bautista and R. Atienza, "Scene Text Recognition with Permuted Autoregressive Sequence Models," in Proc. ECCV, 2022. arXiv:2207.06966
10. A. G. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," 2017. arXiv:1704.04861
11. A. Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in Proc. ICLR, 2021. arXiv:2010.11929
12. S. Mehta and M. Rastegari, "MobileViT: Light-weight, General-purpose, and Mobile-friendly Vision Transformer," in Proc. ICLR, 2022. arXiv:2110.02178
13. S. Mehta and M. Rastegari, "Separable Self-attention for Mobile Vision Transformers," 2022. arXiv:2206.02680
14. U. Pal and B. B. Chaudhuri, "Indian Script Character Recognition: A Survey," Pattern Recognition, vol. 37, no. 9, 2004.
15. M. Mathew, M. Jain and C. V. Jawahar, "Benchmarking Scene Text Recognition in Devanagari, Telugu and Malayalam," in Proc. ICDAR Workshops, 2017.

16. H. Lunia, A. Mondal and C. V. Jawahar, "IndicSTR12: A Dataset for Indic Scene Text Recognition," in Proc. ICDAR, 2023.
17. HarfBuzz text shaping engine. harfbuzz.github.io
18. P. N. Durette, "gTTS: Python library and CLI tool to interface with Google Translate's text-to-speech API." github.com/pndurette/gTTS
19. Office of the Registrar General & Census Commissioner, India, "Census of India 2011: Literacy." censusindia.gov.in
20. United Nations, "The 17 Sustainable Development Goals." sdgs.un.org/goals
21. R. Wightman, "PyTorch Image Models (timm)." github.com/huggingface/pytorch-image-models
22. FastAPI documentation. fastapi.tiangolo.com
23. ONNX Runtime documentation. onnxruntime.ai
24. Android Developers, "CameraX" and "Jetpack Compose" documentation. developer.android.com

Appendix A: Installation and running

Backend

```
cd backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env          (add GEMINI_API_KEY)
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Train the model (Google Colab)

1. Upload ml/ml.zip, or the ml folder, to Google Drive at MyDrive/gramtext/ml.
2. Open ml/train_colab.ipynb in Colab, choose a T4 GPU, and run all cells.
3. Copy the downloaded gramtext_str.onnx to backend/models/ and restart the backend. /api/health then shows str_model_ready: true.

Android app

```
cd android
.\gradlew.bat assembleDebug -PapiBaseUrl=https://<service>.onrender.com
```

The APK is written to android/app/build/outputs/apk/debug/app-debug.apk.

Appendix B: Configuration

Variable	Default	Purpose
GEMINI_API_KEY	(none)	Key for the Gemini backup
GEMINI_MODEL	gemini-flash-lite-latest	Backup model
GEMINI_THINKING	low	Keeps Gemini fast
OCR_ENGINE	auto	auto, str or gemini
STR_MIN_CONFIDENCE	0.85	Below this, use the backup
TTS_TLD	co.in	Indian-English accent
MAX_UPLOAD_MB	10	Largest accepted image
TTS_MAX_CHARS	1500	Longest text for /api/tts

Appendix C: Sample API response

POST /api/ocr with a medicine label photo returns:

```
{
  "success": true,
  "text": "भोजन के बाद दिन में 2 बार\nParacetamol 500 mg",
  "language": "hi",
  "engine": "str",
  "confidence": 0.93,
  "scan_id": 42,
  "message": "Text extracted successfully."
}
```

(Illustrative example of the response format.)

Appendix D: Model code

```

class GramTextSTR(nn.Module):
    def __init__(self, num_classes=NUM_CLASSES, pretrained=False):
        super().__init__()
        self.backbone = timm.create_model(
            "mobilevitv2_050", pretrained=pretrained, features_only=True,
            out_indices=(2,), in_chans=1)
        # Downsample height only, so a 256 px crop keeps 64 time steps
        self.backbone.stages_2[0].conv2_kxk.conv.stride = (2, 1)
        channels = self.backbone.feature_info.channels()[-1]
        self.rnn = nn.LSTM(channels, 128, num_layers=2, bidirectional=True,
                           batch_first=True, dropout=0.1)
        self.head = nn.Linear(256, num_classes)

    def forward(self, x):
        features = self.backbone(x)[-1]
        seq = features.mean(dim=2).transpose(1, 2)
        seq, _ = self.rnn(seq)
        return self.head(seq)

```

Appendix E: Viva preparation

1. **What does GramText do?** It photographs printed Hindi or English labels and notices, recognises the text with our own STR model, and reads it aloud.
2. **Why MobileViT?** It combines convolution for local strokes with transformer blocks for global context, in under 1M parameters, so it is fast on a low-cost server and can later run on phones.
3. **Why CTC?** Word images have no character-position labels. CTC learns the alignment itself and decodes in one fast pass.
4. **How are matras and conjuncts handled?** Recognition works at Unicode code-point level, and training images are shaped with HarfBuzz, so conjuncts render exactly as in print.
5. **Where does the training data come from?** Synthetic rendering of real Hindi and English word lists in 14 fonts with camera-like distortions, plus optional real crops and pilot corrections.
6. **What does Gemini do?** It is a backup when our model's confidence is below 0.85 or the model is missing. Each response says which engine answered, and the metrics track the split.
7. **Why not offline?** It was moved to Phase 2. The model is already small enough for TFLite.
8. **How is impact measured?** Scans, success rate, helpful rate, corrections, latency and unique devices from `/api/metrics`, plus model accuracy from `eval.py` and the feedback forms.
9. **What about privacy?** No images or scanned text are stored. Corrections are sent only with the user's consent.
10. **What about medical safety?** The app only reads text aloud, and every screen carries a disclaimer.